Sharing Without Splitting: Concurrent Conflict Weights for Parallel Crossword Filling

Luke K. Schreiber
Undergraduate Student, Department of Computer Science
Princeton University
Princeton, NJ, USA
ls1881@princeton.edu

Abstract—Parallel CSP solvers typically partition the search space across workers or run an undivided portfolio of independent restarts; prior work treats any sharing between workers as a one-time input to partitioning. This paper describes a restart portfolio whose workers never partition the problem, but continuously and concurrently update a shared pool of per-constraint conflict weights, with no manager and no synchronization phase. We implement it in xfill, a word-level CSP solver for crossword filling, and test it two ways. First, we show restart-portfolio search occupies a genuinely different point in the design space than partition-based search, represented by orca-solver, an independently developed alternative: oversubscribing thread count improves xfill roughly 65 times on one hard grid, while orca-solver improves only marginally and cannot solve it below 14 threads; on 12 real, previously published 15x15 grids, xfill solves every one, and is faster on 9 of the 11 where orca-solver also succeeds, by a geometric mean of 107 times. Partitioning, conversely, gives non-overlapping coverage useful for proving unsatisfiability. Second—this paper's actual contribution—we isolate the mechanism itself, holding solver and grid fixed: a consistent win on moderate-difficulty grids, but higher variance rather than a reliable one on the hardest instances, reported honestly alongside the positive result. We also report a negative result: five plausible branching heuristics produced identical output because a domain-size precondition meant none of them ever executed—caught only by direct instrumentation, not benchmark comparison.

Index Terms—constraint satisfaction, parallel search, restart portfolios, dom/wdeg heuristic, crossword generation

I. Introduction

Crossword filling—assigning a dictionary word to every slot in a grid so every crossing letter agrees—is a word-level constraint satisfaction problem (CSP) and a convenient search-research testbed: instances are human-legible, difficulty is controllable directly through grid geometry, and independently developed solvers already exist to compare against. Ginsberg's Dr.Fill [1] established crossword filling as a serious weighted-CSP application over a decade ago.

This paper is not about crossword filling. It uses crossword filling as a setting to answer one question about parallel constraint search: when multiple workers run concurrently, is there value in sharing soft heuristic statistics—information that biases search order without asserting anything logically—continuously, without ever partitioning the problem those workers are solving? Section II shows this combination does not exist in prior work; Section III implements it; Section V measures its effect, both in isolation and against the broader architectural backdrop that makes it worth measuring at all.

Contributions: (1) a concurrent, decentralized conflict-weight-sharing mechanism for restart-portfolio CSP search, positioned precisely against the closest prior work; (2) a reproducible four-solver testbench that separates two claims—restart-portfolio search as an architectural class versus this specific mechanism within it—rather than conflating them; and (3) a methodological negative result about confirming a heuristic executes before trusting its benchmark comparison.

II. Background and Related Work

A CSP assigns values to variables from finite domains subject to constraints; systematic backtracking assigns one variable at a time, then propagates—removing now-impossible values from other variables' domains—and backtracks if some variable's domain empties entirely (a wipeout). The dom/wdeg (domain-over-weighted-degree) heuristic [4] selects the variable with smallest domain-size-to-weighted-degree ratio, where a constraint's weight increases on every wipeout it causes. Randomized backtracking exhibits heavy-tailed runtime [2], [3]: restarting periodically eliminates most of the tail and is standard in Boolean satisfiability (SAT) and CSP solvers alike.

Two dominant parallel strategies exist. Portfolio approaches run independent searches over the whole problem concurrently; SAT portfolios such as ManySAT [6] additionally share learned clauses (facts inferred from a worker's own failed attempts) between workers without partitioning. Partitioning approaches divide the search space so each worker explores a disjoint region, giving non-overlapping coverage useful for proving unsatisfiability (that no solution exists, UNSAT).

Three prior CSP results sit near, but not on, the point in this design space we describe. Grimes and Wallace [5] accumulate dom/wdeg-style weights from short restarted probes of a single sequential solver—never concurrent. Ehlers and Stuckey [7] parallelize a lazy-clause-generation solver by sharing learnt nogoods (assignments

proven infeasible) and objective bounds between workers—concurrent, but sharing hard facts, and requiring clause-learning machinery most CSP solvers lack. Yun and Epstein's SPREAD [8] is the closest: a manager races workers under a plain dom/wdeg solver, then averages their reported weights once each exhausts a budget, and uses the average to choose how to partition the problem for a subsequent phase. The authors state this directly: "the primary purpose of SPREAD's portfolio phase is to glean information to support search space splitting, not to solve P" [8]. A comprehensive 2018 survey [9] documents extensive concurrent, non-partitioning multi-agent search—sharing hard facts (learned clauses, as in ManySAT) or concrete candidate-value hints—but no work sharing continuously updated soft heuristic weights among such workers.

The mechanism in Section III occupies the gap these leave: concurrent, unlike Grimes and Wallace; soft, unlike Ehlers and Stuckey; and never used to partition, unlike Yun and Epstein.

III. System Architecture

xfill parses a grid into slots (open runs ≥ 2 cells) and crossings (cell-level intersections). Its sequential core performs dom/wdeg-style branching—selecting the slot minimizing domain size over accumulated crossing weight—then propagates letter constraints via an AC-3-style queue (a standard constraint-propagation algorithm); a domain wipeout increases the responsible crossing's weight.

SolveParallel runs N independent restart workers, each with its own seed and private, decaying crossing-weight table, plus one worker dedicated to a single uninterrupted exhaustive search (unlimited_budget) that guarantees eventual completion independent of restart variance. Any worker's sound (mathematically valid) negative result cancels the others.

Concurrent conflict-weight sharing. Alongside each worker's private table, SolveParallel maintains one array of plain atomic counters (thread-safe, uninterruptible updates), one per crossing, shared by every restart worker except the dedicated exhaustive one. Every worker increments a crossing's counter on wipeout and reads every counter when scoring candidates. The counters are plain, order-independent atomic increments rather than a decaying scheme: a decaying normalizer is safe under concurrent writers only if there is just one writer, since it depends on the exact sequence of prior updates—true concurrent writers need an update rule that ignores order entirely. The dedicated exhaustive worker is excluded because its entire value is one undisturbed trajectory; wiring it in measurably hurt performance on the one grid in our corpus it had uniquely been able to solve. Two correctness properties were required: single-threaded calls stay byte-identical to the unshared baseline, since the mechanism only activates for thread count > 1; and ThreadSanitizer (a tool that detects unsafe concurrent memory access), run against real multi-threaded solves, reports no such unsafe access. In earlier development-time testing across a broad corpus of moderate-difficulty grids (documented in docs/design.md), this mechanism cut solve time broadly—a finding we revisit against a harder, targeted ablation in Section V-B.

IV. A Negative Result: Branching on the Wrong Thing

This section is a methodological aside from the same codebase. Motivated by a 7×7 grid known to be exceptionally hard for restart-based search, we implemented five letter-level branching variants—three minimum-remaining-value scoring rules, a faithful reimplementation of a partition-based solver's own candidate scoring, and a reversed scan order—reasoning that a smaller per-node branching factor should help on wide-open grids.

All five produced numerically identical node and backtrack counts—too strong a signal to report as a simple null result, so we checked further. A plain counter in the branch-selection code, tracking how often the large-domain branch these heuristics govern actually executed, showed it firing zero times across more than 17,000 real search nodes: propagation from the grid's pre-filled letters narrowed every domain below the branch's activation threshold within the first few assignments. All five were unreachable code the entire time we compared them, which is why their benchmark scores could not have distinguished them. The actual bottleneck was the small-domain branch's fixed word ordering; extending the solver's existing restart-randomization flag to it produced the improvements reported in Section V-C. The reusable point: an A/B comparison of branching heuristics is only informative once the branch under test is confirmed, by direct instrumentation, to execute.

V. Experiments

A. Setup

The experiments below—enabled by Section IV's fix—test two distinct claims, kept in separate subsections rather than blended together. Section V-B isolates the mechanism itself first, holding architecture fixed, since that is this paper's actual contribution. Sections V-C–V-F then test the broader architectural claim from the Introduction directly, using orca-solver as a concrete, independently developed stand-in for partition-based search, and an unmodified naive backtracker to show the comparison is not trivial. That architectural comparison is context for why the mechanism is worth building, not a prerequisite to testing it.

Solvers. xfill (this work); orca-solver, a partition-based parallel solver treated as a black box; ingrid_core, a single-threaded reference solver; and, as a naive baseline with no restarts, no learned heuristics, and no parallelism, crossword-composer (a static-constraint-order Rust backtracker). All four run through a single fixed harness

(benchmarks/urtc2026_testbench/) in the paper's code repository [11], against a dictionary derived deterministically from its own freely available word list, with a fixed seed and a 300-second cap, reproducible via run_benchmark.py; every timeout was re-tested at 600s, then again at 1000s, via rerun_timeouts.py to rule out an under-provisioned cap (Section V-E). xfill and orca-solver race independent workers, so wall time varies run to run; each runs 5 times per grid, median reported, every trial logged. ingrid_core and crossword-composer are single-threaded and deterministic, so each runs once.

Grid set. A size-graded curated set (5×5 through 21×21) plus a fixed-seed random sample of 15×15 grids scraped from a public corpus of real, previously published crossword layouts, crosswordgrids.com [10].

B. Isolating the Mechanism: A Shared-Weight Ablation

This section tests xfill against itself: same solver, same grid, same dictionary, same 14 threads, one environment variable flipped, XFILL_DISABLE_SHARED_WEIGHTS on and off, several trials each, on grids from two independent sources—a fixed "known-hard" grid list reused from this project's own earlier development-time benchmarking (not cherry-picked for this paper; the same list anchors the thread-count scaling result in Section V-C), and three grids from the paper's own main scraped-15×15 corpus, to check the result is not an artifact of one curated list.

Fig. 1 does not show a uniform win, on either grid set, and we report that plainly. On grid_053, the unshared baseline is faster on the median (0.72s vs. 2.77s) and far tighter (0.66–0.76s vs. 0.07–3.71s). On grid_058 and grid_479, medians are similar but shared weights show wider spread. On grid_120, shared weights reach a lower best case but also produce the only outright timeout among those eighteen trials. grid_395 and grid_423 sharpen both directions: a clean $\sim$7$\times$ win on the former (36.9s vs. 271.4s median), and actively worse on the latter—all 5 trials hit the 300s cap, where disabling shared weights solves it reliably at $\sim$180s every time. grid_423 reappears in Section V-F as the grid where xfill uniquely completes what orca-solver cannot—a completion advantage this ablation shows rests on exactly the unreliable configuration: shared weights on, the default. Against Section III's broader-corpus finding of a consistent win, the synthesis is regime-dependent. Shared weights help on typical grids by redistributing which crossings each worker deprioritizes, but at the extreme difficulty end they trade tight, well-behaved variance for a wider one that is not reliably better, and sometimes reliably worse. We have no evidence for a uniform win, so we do not claim one—the same standard Section IV applies to the heuristics it rules out.

C. Restart Portfolios Exploit Added Parallelism

Restart-portfolio search should benefit from more independent workers even past the physical core count, because heavy-tailed runtime means each additional worker is another draw that might get lucky early. We tested this directly on three grids known to be hard for restart-based search, oversubscribing threads well past this machine's 14 physical cores (Fig. 2).

D. Is This a Property of the Architecture, or One Grid?

Fig. 2's result splits three ways, showing this is a property of the architecture rather than a lucky pick of grid: on grid_120, xfill's wall time falls from 27.8s at 1 thread to 0.43s at 42—a 65$\times$ improvement—while orca-solver never solves it below 14 threads and only creeps from 27.3s to 17.9s from 14 to 42; on grid_303, orca-solver fails to solve it at any of the 7 thread counts tested, while xfill solves it (noisily, not monotonically) at 8, 14, and 42 threads; on grid_115, neither solver succeeds at any thread count—this one favors neither architecture. Where the pattern does hold, it holds because absorbing extra parallelism without restructuring the search is what a non-partitioning restart portfolio is built to do; a partition-based design here does not exhibit that capacity to nearly the same degree.

E. Naive Backtracking Is Not a Baseline Worth Beating

Zooming out from thread scaling on a few hand-picked hard grids to the full sweep across all four solvers and every curated grid size (Fig. 3), the same advantage holds broadly, letting us test directly whether crossword-composer's losses are a genuine baseline failure or just an under-provisioned time budget. Across all 20 grids, xfill resolved 19 (18 solved, 1 UNSAT; 1 timeout), orca-solver 18 (17 solved, 1 UNSAT; 2 timeouts), ingrid_core 17 (16 solved, 1 UNSAT; 3 timeouts), and crossword-composer 9 (8 solved, 1 UNSAT; 11 timeouts)—a real, unmodified, independently authored solver, not a strawman built to lose. Escalating the cap from 300s to 600s changed exactly one outcome: ingrid_core went on to solve grid_479 in 325s. A further escalation to 1000s—the cap these figures use—changed none: every remaining timeout, all 11 of crossword-composer's plus sample_13x13, the one grid no solver resolves at any cap tested, held again: evidence these are hard, not under-budgeted. The point for this paper's thesis is narrower than "xfill wins": at the grid and dictionary sizes crossword filling actually requires, restarts, adaptive heuristics, and parallelism are not optional refinements—they are the premise on which the rest of this section, and the mechanism this paper contributes, depend.

F. Aggregate Speed Advantage Where Both Solve

Fig. 4 quantifies the restart-portfolio/partition comparison on the scraped 15×15 corpus alone [10]—realistic-scale grids, rather than the curated size-graded set, which mixes

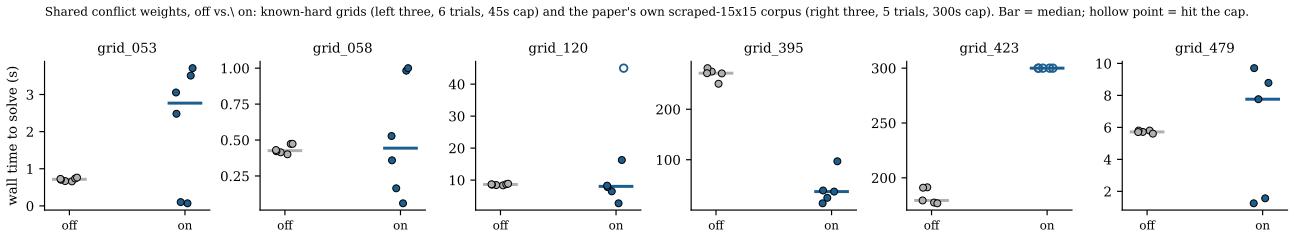

Fig. 1. Shared-conflict-weight ablation, off vs. on, 14 threads: a known-hard grid list also used in Fig. 2 (left three, 6 trials, 45s cap) and three grids from the paper's own scraped-15 × 15 corpus (right three, 5 trials, 300s cap; the other 9 of 12 solve in under a second and are omitted). Bars mark the median; hollow points hit the cap. Shared weights are not uniformly better: the same win/wash/loss pattern appears independently on both grid sets.

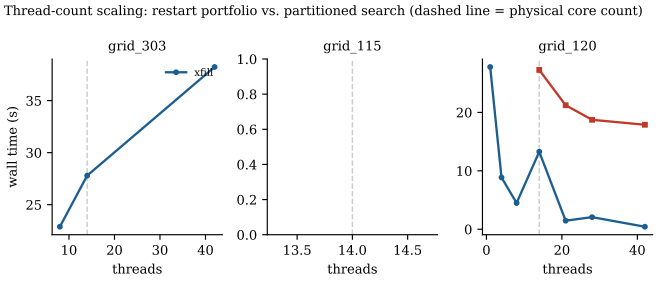

Fig. 2. Wall time vs. thread count, xfill vs. orca-solver, on three grids known to be hard for restart-based search (45s cap; orca-solver missing from a panel means every one of its 7 thread counts timed out). grid_120 is the clearest case: xfill improves roughly 65× from 1 to 42 threads while orca-solver stays flat and cannot solve at all below 14.

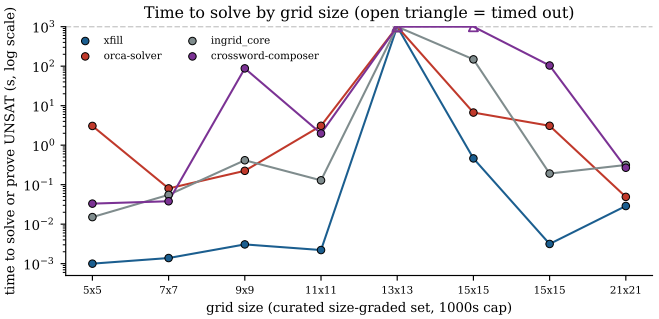

Fig. 3. Time to solve or prove UNSAT by grid size, curated set, 1000s cap (log scale; open triangle = timed out). A solved/not-solved view of this same data would show xfill, orca-solver, and ingrid_core essentially tied, since all three succeed on the same grids here—the actual difference is that xfill's line sits one to three orders of magnitude below the others at every size except 13 × 13 (every solver times out together) and 21 × 21 (an easy UNSAT proof every solver resolves in well under a second regardless).

in small grids that inflate the ratio. Of the 11 scraped grids (of 12) both xfill and orca-solver solve, xfill is faster on 9, with a median speedup of 494× and a geometric mean of 107×. Two exceptions run the other way: on grid_395 and grid_479, orca-solver is faster, by 5.4× and 1.6×—consistent with grid_115 in Section V-D: not every grid favors either architecture. grid_423, the twelfth scraped grid, sharpens this in the opposite direction: xfill solves it in 270s where orca-solver cannot, even at this paper's full 1000s cap—a completion advantage, not just a speed one (revisited in Section V-B). Across the full 20-grid sample, xfill resolves 19, orca-solver 18, both agree on the sample's one correct UNSAT proof, and xfill's one failure (sample_13x13) is a subset of orca-solver's two, grid_423 being the other—a narrower gap on success alone than on speed, which is why Sections V-D and V-B both turn to a harder, pre-existing grid list for a more demanding test.

VI. Discussion

The architectural comparison and the mechanism ablation share one underlying cause: restart portfolios benefit from heavy-tailed runtimes, so more independent draws raise the chance of an early lucky one. This is why oversubscribing thread count helped in Section V-C and Fig. 2, and why grid_423 (Fig. 4) goes further: more draws can succeed where a fixed partition cannot. Partitioned search instead gives every worker a disjoint slice of the space to rule out, which is exactly what makes it valuable for proving unsatisfiability (Section II). The same heavy-tailedness explains Fig. 1's mixed result: shared conflict weights change which draw from the tail each worker takes, which can land on a worse draw as easily as a better one, netting out as an improvement only once averaged over a broader, typical corpus.

VII. Limitations and Conclusion

All timing results are from one 14-core machine and one dictionary derived from a specific freely available word list; the thread-scaling numbers should not be read as hardware- or dictionary-independent claims. Both principal solvers are actively developed; exact commit hashes are recorded in the testbench README.

This paper's contribution is the mechanism in Section III: concurrent conflict-weight sharing across a restart portfolio that never partitions, occupying a gap that three adjacent lines of prior work leave open. The architectural comparison in Section V supports that gap's importance rather than being the paper's subject—restart portfolios and partitioned search are suited respectively to finding solutions quickly and to proving none exist, a pattern

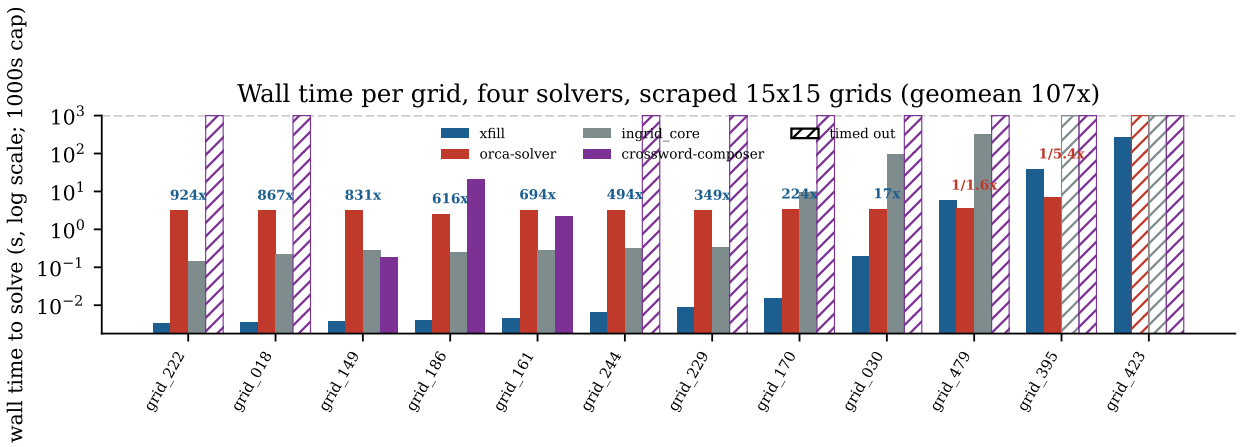

Fig. 4. Wall time to solve (log scale, 1000s cap), four solvers, per scraped 15×15 grid, sorted by xfill's time. Hatched bars mark a timeout; crossword-composer times out on most grids, the same pattern reported by count in Section V-E. Label above the xfill/orca-solver pair is orca-solver time / xfill time; two grids (grid_395, grid_479) favor orca-solver, and on grid_423 orca-solver cannot solve it even at this cap while xfill does, in 270s.

independent of any one solver. The mechanism itself, tested in isolation, is more modest than the architecture it sits inside: a real win on typical grids, a wash or worse on the hardest ones, reported as measured rather than as hoped. We also reported a negative result from the same codebase—five heuristics rendered indistinguishable by a precondition that kept them from ever executing—as a reusable caution for comparing branching heuristics by benchmark score alone. Nothing about the mechanism itself is specific to crossword filling: it needs only a dom/wdeg-style weight table and a restart loop, both already common in CSP and lazy-clause-generation solvers—exactly the ingredients Section II found no prior system combining this way.

Acknowledgment

This work received no external funding or advisor supervision. The author used Claude Code (Anthropic), an AI coding assistant, throughout the project: to help implement and test the xfill solver, to build the benchmarking infrastructure and run the experiments reported here, to generate the figures, and to draft and revise this manuscript. The author reviewed, verified, and takes full responsibility for all code, experimental results, and conclusions in this paper.

References

[1] M. L. Ginsberg, "Dr.Fill: crosswords and an implemented solver for singly weighted CSPs," Journal of Artificial Intelligence Research, vol. 42, pp. 851–886, 2011.

[2] C. P. Gomes, B. Selman, and H. Kautz, "Boosting combinatorial search through randomization," in Proc. AAAI, 1998, pp. 431–437.

[3] C. P. Gomes, B. Selman, N. Crato, and H. Kautz, "Heavy-tailed phenomena in satisfiability and constraint satisfaction problems," Journal of Automated Reasoning, vol. 24, pp. 67–100, 2000.

[4] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais, "Boosting systematic search by weighting constraints," in Proc. ECAI, 2004, pp. 146–150.

[5] D. Grimes and R. J. Wallace, "Sampling strategies and variable selection in weighted degree heuristics," in Proc. CP, LNCS vol. 4741, 2007, pp. 831–838.

[6] Y. Hamadi, S. Jabbour, and L. Sais, "ManySAT: a parallel SAT solver," J. Satisfiability, Boolean Modeling and Computation, vol. 6, pp. 245–262, 2009.

[7] T. Ehlers and P. J. Stuckey, "Parallelizing constraint programming with learning," in Proc. CPAIOR, LNCS vol. 9676, Banff, AB, Canada, 2016, pp. 142–158.

[8] X. Yun and S. L. Epstein, "A hybrid paradigm for adaptive parallel search," in Proc. CP, LNCS vol. 7514, 2012, pp. 720–736.

[9] I. P. Gent, I. J. Miguel, P. W. Nightingale, C. McCreesh, P. Prosser, N. Moore, and C. Unsworth, "A review of literature on parallel constraint solving," Theory and Practice of Logic Programming, vol. 18, no. 5–6, pp. 725–758, 2018.

[10] Crosswordgrids.com. "15x15 crossword grids." Accessed: Jul. 29, 2026. [Online.] Available: https://crosswordgrids.com/15x15-crossword-grids

[11] L. K. Schreiber. xfill: a word-level CSP solver for crossword filling. (2026). Source code, benchmarks, and results. Accessed: Aug. 7, 2026. [Online]. Available: https://github.com/ls1881/crossword-filler